

多媒体第四次实验

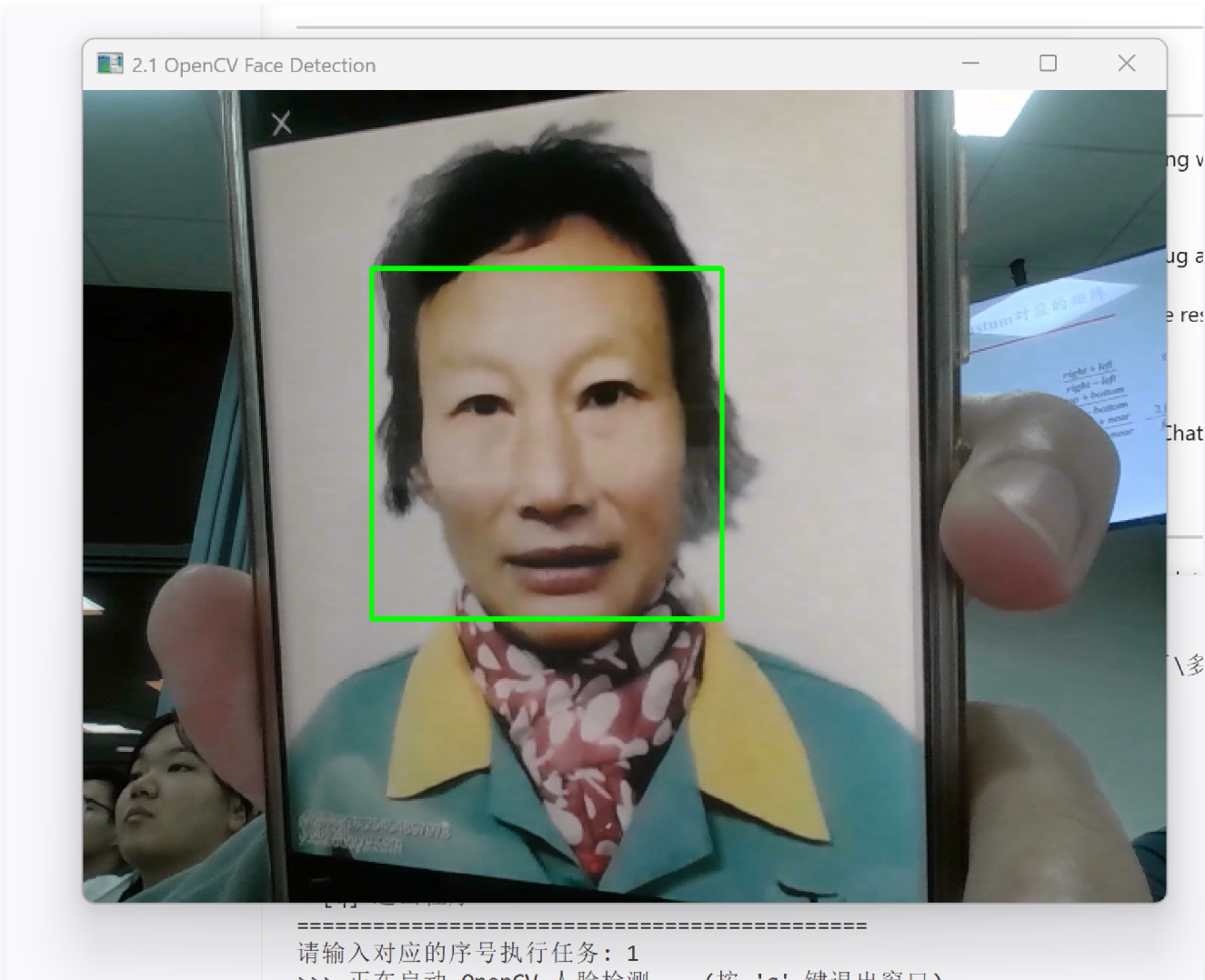
1. 完成本次实验任务的思路

本次实验的核心目标是掌握视频流的实时捕获及基于机器视觉的面部特征检测技术。具体实现思路分为两套技术路线：

- **OpenCV 视频捕获与人脸检测任务思路：** 第一套路线基于传统机器学习中的 Haar 级联分类器。首先，通过 `cv2.VideoCapture(0)` 接口建立与本地硬件摄像头的数据连接，并利用 `while` 循环机制逐帧拉取视频流。考虑到 Haar 特征评估对计算性能的要求，需将彩色的 BGR 视频帧转化为单通道灰度图，以降低数据维度并提升匹配速度。最后，调用预加载的 `haarcascade_frontalface_default.xml` 模型，利用多尺度检测方法（`detectMultiScale`）在灰度帧中搜索人脸区域，获取边界坐标，并通过 `cv2.rectangle` 函数在原始真彩视频帧上实时叠加矩形边界框进行可视化。
- **dlib 68 个人脸特征点检测任务思路：** 第二套路线引入 dlib 库以实现更高精度的细粒度面部特征提取。该路线在获取视频帧并转换为灰度图后，采用“先检测后预测”的两阶段架构：首先，利用 dlib 基于方向梯度直方图（HOG）与线性 SVM 构建的正脸检测器，精确推断出人脸的物理边界盒（Bounding Box）；随后，将该边界盒区域作为感兴趣区域（ROI）输入至基于回归树集合的特征点预测器中（依赖 `shape_predictor_68_face_landmarks.dat` 模型），提取出人脸五官及轮廓的 68 个关键二维像素坐标。最后，遍历这 68 个点位集合，利用 `cv2.circle` 函数在画面中渲染实心圆点，实现关键点的实时动态跟踪。

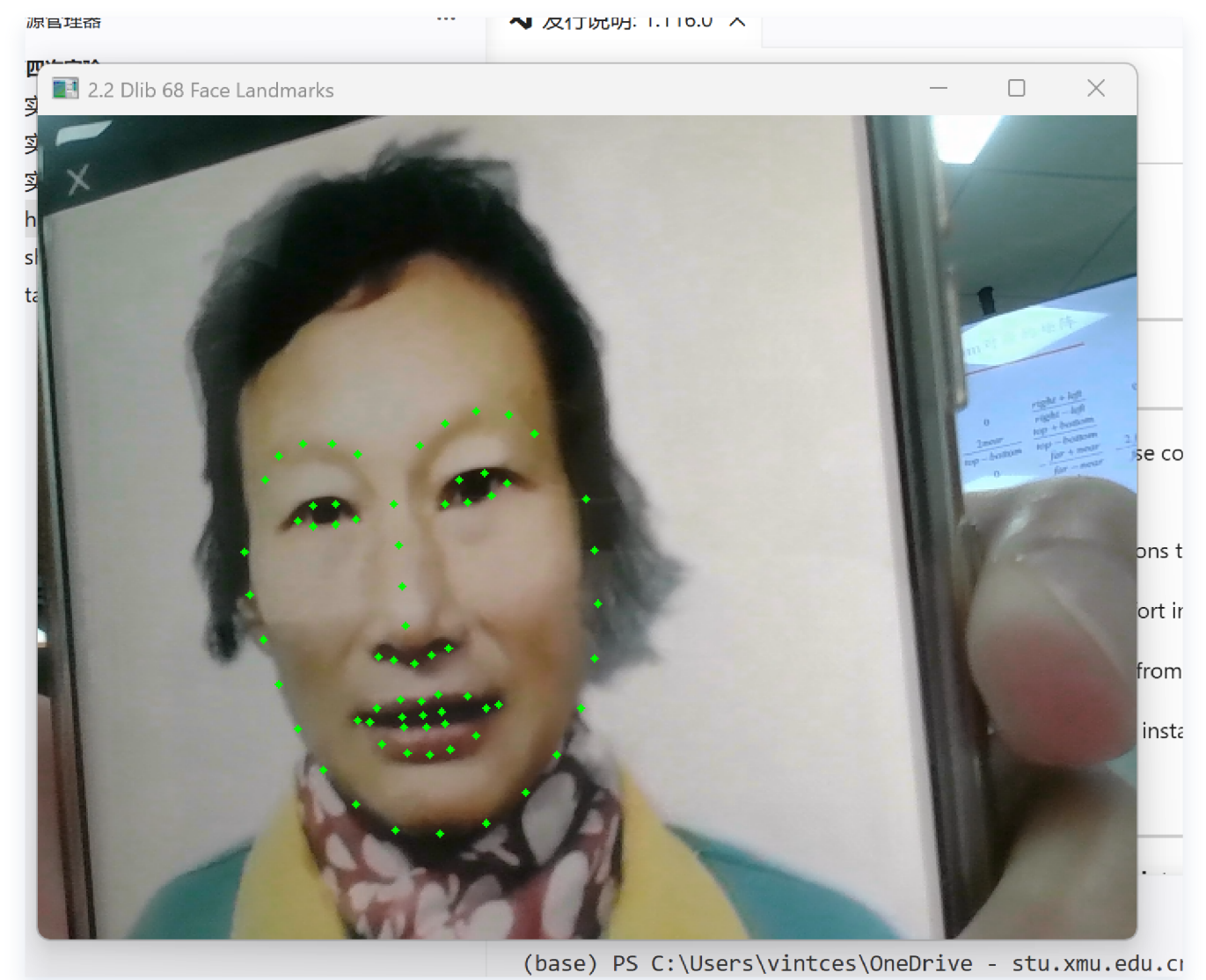
2. 运行程序截图和简要说明

2.1 OpenCV 实时视频人脸检测效果



说明：程序成功初始化并调用了本地摄像头，能够以较高的帧率流畅读取视频流。如图所示，当画面中出现人物面部时，基于 Haar 特征的 OpenCV 级联分类器能够迅速捕捉到面部轮廓，并利用亮绿色的矩形边框（Bounding Box）对面部区域进行准确的框选和持续跟踪。该算法对正脸识别率高，且能够在人物轻微晃动或背景存在一定干扰时，依然保持良好的识别鲁棒性和极低的画面延迟。

2.2 dlib 68个人脸特征点检测效果



说明：如图所示，dlib 模块不仅成功识别出了画面中的人脸位置，更进一步地在人脸的精细解剖结构上（包括眉毛、眼睛、鼻梁、嘴唇边缘以及整个下颌线轮廓）精准覆盖了 68 个绿色特征点。相比于 OpenCV 的单一粗略矩形框，该方法能够清晰地反映出人物当前的局部特征与面部姿态细节，实现了高精度的细粒度人脸关键点标定，且点位在视频流中能够稳定地贴合面部轮廓线移动。

3. 核心代码展示和分析

3.1 OpenCV 视频捕获与人脸框检测逻辑

```
def detect_face_opencv():  
    """  
    任务一：利用 OpenCV 实时捕获视频并进行人脸矩形框检测  
    要求：实时读取视频流，使用 opencv 检测人脸并画矩形框，最后实时显示。  
    """
```

```
print(">>> 正在启动 OpenCV 人脸检测... (按 'q' 键退出窗口)")

# 1. 加载级联分类器文件
try:
    face_cascade = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
    if face_cascade.empty():
        raise IOError("无法加载 haarcascade_frontalface_default.xml, 请检查文件路
径! ")
except Exception as e:
    print(f"错误: {e}")
    return

# 2. 打开默认摄像头
cap = cv2.VideoCapture(0)
if not cap.isOpened():
    print("错误: 无法打开摄像头! ")
    return

while True:
    ret, frame = cap.read()
    if not ret:
        print("无法获取画面流, 正在退出...")
        break

    # 转为灰度图, 提高检测速度和准确率
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # 进行人脸检测
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1,
minNeighbors=5, minSize=(30, 30))

    # 3. 遍历检测到的人脸并画绿色矩形框
    for (x, y, w, h) in faces:
        cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)

    # 实时显示在 opencv 窗口中
    cv2.imshow('2.1 OpenCV Face Detection', frame)

    # 按 'q' 键退出循环
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

# 释放资源并销毁窗口
cap.release()
```

```
cv2.destroyAllWindows()
```

代码分析：本段代码实现了基于 OpenCV 的基础实时视频流处理及人脸识别管线。代码首先通过异常捕获机制（try-except）安全加载了 XML 模型，保证了程序在文件缺失时的健壮性。进入 `while` 循环后，`cap.read()` 负责源源不断地抽取图像帧。核心的预处理操作 `cv2.cvtColor` 将彩色帧转换为灰度图，以满足 Haar 分类器的底层灰度特征运算需求。`face_cascade.detectMultiScale` 是目标检测的核心执行者，其中的 `scaleFactor=1.1` 和 `minNeighbors=5` 参数有效平衡了多尺度检测的漏检率与误报率。最终返回的坐标元组经由 `cv2.rectangle` 函数消费，将几何边框实时反绘于原始画面中。

3.2 dlib 68个人脸特征点提取与绘制逻辑

```
def detect_landmarks_dlib():
    """
    任务二：利用 dlib 检测视频流中的 68 个人脸特征点
    要求：实时读取视频流，使用 dlib 检测特征点并画在图像帧上，最后实时显示。
    """

    print(">>> 正在启动 dlib 68 个人脸特征点检测... (按 'q' 键退出窗口)")
    print(">>> 提示：dlib 加载模型可能需要几秒钟，请稍候...")

    # 1. 初始化 dlib 的人脸检测器和特征点预测器
    try:
        detector = dlib.get_frontal_face_detector()
        predictor = dlib.shape_predictor('shape_predictor_68_face_landmarks.dat')
    except RuntimeError as e:
        print(f"错误：无法加载 shape_predictor_68_face_landmarks.dat，请确认文件是否存在！\n详细信息：{e}")
        return

    # 2. 打开默认摄像头
    cap = cv2.VideoCapture(0)
    if not cap.isOpened():
        print("错误：无法打开摄像头！")
        return

    while True:
        ret, frame = cap.read()
        if not ret:
            print("无法获取画面流，正在退出...")
            break

        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```
# 使用 dlib 检测人脸位置
faces = detector(gray, 0)

# 3. 对每个检测到的人脸提取 68 个特征点
for face in faces:
    landmarks = predictor(gray, face)

    # 遍历 68 个点，并在图像上绘制实心圆
    for n in range(0, 68):
        x = landmarks.part(n).x
        y = landmarks.part(n).y
        cv2.circle(frame, (x, y), 2, (0, 255, 0), -1)

# 实时显示在 opencv 窗口中
cv2.imshow('2.2 Dlib 68 Face Landmarks', frame)

# 按 'q' 键退出循环
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

# 释放资源并销毁窗口
cap.release()
cv2.destroyAllWindows()
```

代码分析：本段代码展示了 dlib 库在复杂特征提取中的“级联推断”逻辑。初始化阶段，程序分别实例化了人脸边界检测器（`detector`）与 68 点位回归预测器（`predictor`）。在主循环中，程序首先调用 `detector(gray, 0)` 获取灰度图像中的面部感兴趣区域（ROI）。接着，通过循环将获取到的 `face` 包围盒传入 `predictor` 中执行高精度坐标推断，返回的 `landmarks` 对象内完整封装了 68 个关键点的数据。最终，通过 `range(0, 68)` 循环遍历，利用 `.part(n).x` 与 `.part(n).y` 逐一剥离出具体的像素坐标，并使用 `cv2.circle` 以实心圆（厚度参数为 -1）的形式完成点位的图像层叠渲染。